Name:	**Jay Chandrasen Talekar**
Roll No:	**50**
Class/Sem:	TE/VI
Experiment No.:	08
Title:	To deploy a serverless function using AWS Lambda.
Date of Performance:	
Date of Submission:	
Marks: **Sign of Faculty:**	

Aim

To deploy a serverless function using AWS Lambda.

Theory

AWS Lambda

AWS Lambda is a serverless, event-driven compute service provided by Amazon Web Services (AWS). It allows users to run code without provisioning or managing servers. The function executes in response to events, and AWS manages the underlying infrastructure.

Lambda supports several programming languages, including Python, Node.js, Java, and others. It is useful for building web applications, APIs, automation workflows, and event-processing systems.

Serverless Computing

Serverless computing is a cloud-computing model in which the cloud provider manages the servers, operating systems, and infrastructure required to execute applications. Developers focus on writing and deploying code rather than managing servers.

AWS Lambda Function

A Lambda function is a piece of code that performs a specific task when invoked. It generally consists of:

1. Function name: A unique name used to identify the function.
2. Runtime: The programming language and execution environment used by the function.
3. Handler: The function that AWS Lambda invokes when the function runs.
4. Execution role: An IAM role that defines the AWS permissions available to the function.
5. Event: Input data that triggers or is passed to the function.
6. Return value: The result produced by the function.

Advantages of AWS Lambda

- No need to manage or maintain servers.
- Automatic scaling according to incoming requests.
- Pay-per-use pricing model.
- Easy integration with other AWS services.
- Suitable for event-driven applications.
- Supports multiple programming languages.

Steps

1. Sign in to AWS Management Console

Open the AWS Management Console (https://aws.amazon.com/console/) and sign in using an AWS account.

2. Open AWS Lambda

Search for Lambda in the AWS Management Console.

Select the Lambda service.

Choose the appropriate AWS Region from the region selector.

3. Create a Lambda Function

Click Create function.

Select Author from scratch.

Enter Function name: MyFirstLambdaFunction.

Select Runtime: Python 3.x.

Keep the default architecture.

Select Create a new role with basic Lambda permissions.

Click Create function.

4. Write the Lambda Function Code

In the Code source section, open lambda_function.py and replace its contents with the following code:

```
import json

def lambda_handler(event, context):
    message = {
        "statusCode": 200,
        "body": json.dumps(
            "Hello from AWS Lambda!"
        )
    }

    return message
```

The lambda_handler function is the entry point for the Lambda function. The event parameter contains input data, while context provides information about the invocation and execution environment.

5. Deploy the Function

7. Click the Deploy button in the Lambda console.
8. Wait for the confirmation that the code has been updated successfully.
9. The function is now deployed and ready to be invoked.

6. Create a Test Event

10. Click the Test tab or choose Create test event.
11. Select a suitable event template, such as a basic Hello World event.
12. Enter an event name, for example, MyTestEvent.
13. Use the following sample event:

```
{
  "name": "Student"
}
```

Save the test event.

7. Invoke and Test the Function

14. Select the test event created in the previous step.
15. Click Test.

16. AWS Lambda executes the deployed function.
17. Observe the execution result, response, and logs displayed in the console.

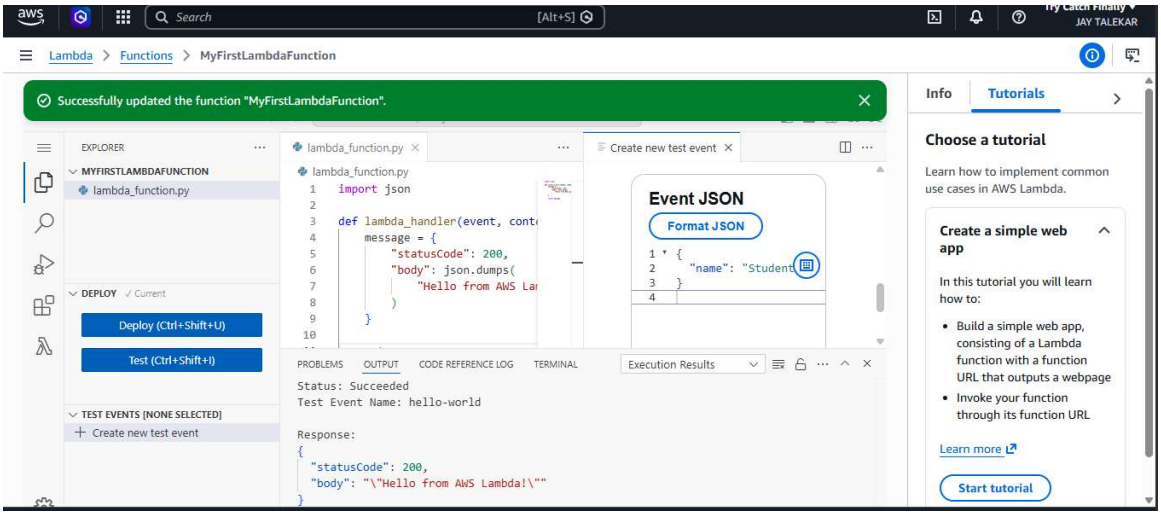
8. Observe the Output

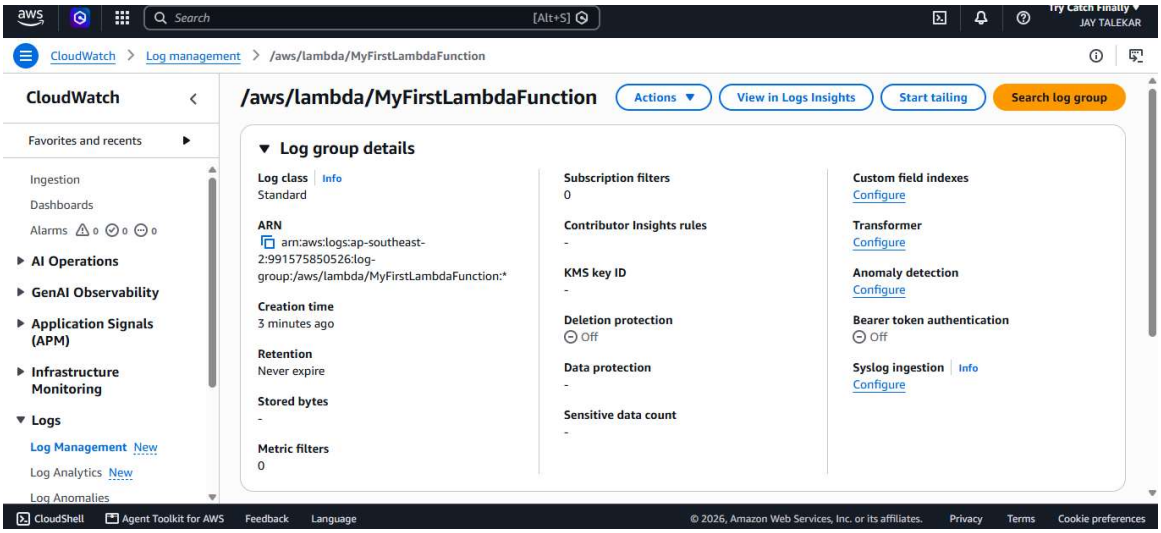
The expected response is similar to:

```
{
  "statusCode": 200,
  "body": "\"Hello from AWS Lambda!\""
}
```

The execution result should indicate that the function completed successfully.

Output





Conclusion

The concepts of serverless computing were studied, and a serverless function was created, deployed, and tested using AWS Lambda. The function was successfully invoked through the AWS Management Console.

Questions

1. What is AWS Lambda ?

   AWS Lambda is a serverless computing service provided by AWS that allows users to run code without managing or maintaining servers.

2. What is meant by serverless computing?

   Serverless computing is a cloud-computing model in which the cloud provider manages the servers and infrastructure. The developer only needs to write and deploy the code.

3. What is the purpose of the Lambda handler?

The **Lambda handler** is the main function that AWS Lambda executes when the function is invoked. Example : def lambda_handler(event, context):

4. What is an execution role in AWS Lambda?

   An **execution role** is an IAM role that gives a Lambda function permission to access AWS services and resources required during execution.

5. What is the use of the event parameter?

The **event parameter** contains the input data or information sent to the Lambda function when it is invoked. Example:{ "name": "Student"}

6. What are the advantages of AWS Lambda?

The main advantages are:

- No need to manage servers.
- Automatically scales according to demand.
- Pay-per-use model.
- Easy integration with other AWS services.
- Supports multiple programming languages.
- Suitable for event-driven applications.

7. How is a Lambda function invoked?

A Lambda function can be invoked through the **AWS Console, API calls, AWS services, event triggers, AWS CLI, or SDKs**. In this practical, the function is invoked using the **Test** button in the AWS Lambda Console.

8. What is the difference between traditional server-based computing and serverless computing?

Traditional Server-Based Computing	**Serverless Computing**
User manages the server.	AWS manages the server.
Infrastructure needs to be configured.	No server configuration is required.
User manages scaling.	AWS automatically handles scaling.
Server may run continuously.	Code runs when it is invoked.
More server maintenance is required.	Less infrastructure maintenance is required.

In simple words: Traditional computing requires you to manage the server, while serverless computing allows you to focus mainly on your code while AWS manages the underlying infrastructure.